

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems

COURSE CODE: CSA0509

COURSE OUTCOME COVERED

CO3: *Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity. (BL2, BL3)*

Activity Tool : Case Study (Medium)

Assessment Tool Weightage: 30%

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Analyze the case: An online shopping platform stores Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price). Identify FDs and normalize to 3NF.	BL2 – Understand	5
2	A hospital maintains Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName). Identify anomalies and normalize to BCNF.	BL3 – Apply	5
3	Case study: A university stores Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade). Analyze FDs and apply 2NF decomposition.	BL3 – Apply	5
4	Given the airline booking case: Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date). Normalize up to BCNF.	BL3 – Apply	5
5	Analyze an HR system schema for a multinational company. Identify all functional and transitive dependencies and normalize to 3NF.	BL2 – Understand	5
6	Case: Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate). Identify MVDs and decompose to 4NF.	BL3 – Apply	5

7	Analyze a telecom billing schema and identify all candidate keys, primary keys, and functional dependencies before normalization.	BL2 – Understand	5
8	A retail inventory system has Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse). Apply complete normalization.	BL3 – Apply	5
9	Study the given poorly normalized schema for a School ERP and propose a fully normalized design up to BCNF with justification.	BL3 – Apply	5
10	Given a movie streaming platform schema, identify join dependencies and decompose to 5NF with lossless-join verification.	BL3 – Apply	5

Code of Conduct:

*I **Shaik Ayesha Tabassum(192525074)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: S.Ayesha Tabassum

RUBRICS FOR CALCULATION:

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	Thorough understanding ; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding ; some issues missed	Poor understanding ; problem not identified correctly
Application of Concepts	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning

Solution Design	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

1. Analyze the case: An online shopping platform stores Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price). Identify FDs and normalize to 3NF.

```
mysql> CREATE DATABASE OnlineShopping;
Query OK, 1 row affected (0.05 sec)

mysql> use onlineshopping;
Database changed
mysql> CREATE TABLE Customer ( CustID VARCHAR(10) PRIMARY KEY, CustName VARCHAR(50) NOT NULL );
Query OK, 0 rows affected (0.20 sec)

mysql> CREATE TABLE Product ( ProdID VARCHAR(10) PRIMARY KEY, ProdName VARCHAR(50) NOT NULL, Price DECIMAL(10,2) NOT NULL );
Query OK, 0 rows affected (0.13 sec)

mysql> CREATE TABLE Order_Details ( OrderID VARCHAR(10), CustID VARCHAR(10), ProdID VARCHAR(10), Qty INT NOT NULL, PRIMARY KEY (OrderID, ProdID), FOREIGN KEY (CustID) REFERENCES Customer(CustID), FOREIGN KEY (ProdID) REFERENCES Product(ProdID) );
Query OK, 0 rows affected (0.11 sec)

mysql> INSERT INTO Customer VALUES ('C01', 'sonu'), ('C02', 'pranathi'), ('C03', 'roops');
Query OK, 3 rows affected (0.05 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Product VALUES ('P01', 'Laptop', 50000), ('P02', 'Mouse', 1000), ('P03', 'Keyboard', 2000);
Query OK, 3 rows affected (0.06 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Order_Details VALUES ('O101', 'C01', 'P01', 1), ('O101', 'C01', 'P02', 2), ('O102', 'C02', 'P01', 1), ('O103', 'C03', 'P03', 1);
Query OK, 4 rows affected (0.09 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM Customer;
+-----+-----+
| CustID | CustName |
+-----+-----+
| C01    | sonu     |
| C02    | pranathi |
| C03    | roops    |
+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT * FROM Product;
+-----+-----+-----+
| ProdID | ProdName | Price |
+-----+-----+-----+
| P01    | Laptop   | 50000.00 |
| P02    | Mouse    | 1000.00  |
| P03    | Keyboard | 2000.00  |
+-----+-----+-----+
3 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM Product;
+-----+-----+-----+
| ProdID | ProdName | Price |
+-----+-----+-----+
| P01    | Laptop   | 50000.00 |
| P02    | Mouse    | 1000.00  |
| P03    | Keyboard | 2000.00  |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT * FROM Order_Details;
+-----+-----+-----+-----+
| OrderID | CustID | ProdID | Qty |
+-----+-----+-----+-----+
| O101    | C01    | P01    | 1   |
| O101    | C01    | P02    | 2   |
| O102    | C02    | P01    | 1   |
| O103    | C03    | P03    | 1   |
+-----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql> SELECT o.OrderID, c.CustID, c.CustName, p.ProdID, p.ProdName, o.Qty, p.Price FROM Order_Details o JOIN Customer c ON o.CustID = c.CustID JOIN Product p ON o.ProdID = p.ProdID;
+-----+-----+-----+-----+-----+-----+-----+
| OrderID | CustID | CustName | ProdID | ProdName | Qty | Price |
+-----+-----+-----+-----+-----+-----+-----+
| O101    | C01    | sonu     | P01    | Laptop   | 1   | 50000.00 |
| O101    | C01    | sonu     | P02    | Mouse    | 2   | 1000.00  |
| O102    | C02    | pranathi | P01    | Laptop   | 1   | 50000.00 |
| O103    | C03    | roops    | P03    | Keyboard | 1   | 2000.00  |
+-----+-----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

Given Relation

Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price)

Step 1: Identify Functional Dependencies (FDs)

Assuming:

- Each customer has one unique name.
- Each product has one fixed name and price.
- An order can contain multiple products.

The functional dependencies are:

1. **CustID** → **CustName**
2. **ProdID** → **ProdName, Price**
3. **(OrderID, ProdID)** → **Qty**

4. **OrderID** → **CustID**

From these, we can also derive:

- **OrderID** → **CustName** (via CustID)
- **(OrderID, ProdID)** → **CustID, CustName, ProdName, Price, Qty**

Step 2: Candidate Key

Since one order may contain multiple products, the composite key is:

Primary Key = (OrderID, ProdID)

Normalization

First Normal Form (1NF)

The relation is already in **1NF** because:

- All attributes contain atomic values.
- No repeating groups.

Second Normal Form (2NF)

Remove partial dependencies:

- CustName depends only on CustID.
- ProdName and Price depend only on ProdID.
- CustID depends only on OrderID.

Decompose into:

Customer

- CustID (PK)
- CustName

Product

- ProdID (PK)
- ProdName
- Price

Order

- OrderID (PK)
- CustID (FK)

OrderItem

- OrderID (FK)
- ProdID (FK)
- Qty
- **Primary Key:** (OrderID, ProdID)

Third Normal Form (3NF)

Check for transitive dependencies:

- In **Customer**, $\text{CustID} \rightarrow \text{CustName}$ (no transitive dependency).
- In **Product**, $\text{ProdID} \rightarrow \text{ProdName}, \text{Price}$ (no transitive dependency).
- In **Order**, $\text{OrderID} \rightarrow \text{CustID}$ (no non-key attribute determines another non-key attribute).
- In **OrderItem**, $(\text{OrderID}, \text{ProdID}) \rightarrow \text{Qty}$ (fully dependent on the key).

Therefore, all relations satisfy **3NF**.

Final 3NF Schema

Customer

CustID (PK) CustName

Product

ProdID (PK) ProdName Price

Order

OrderID (PK) CustID (FK)

OrderItem

OrderID (FK) ProdID (FK) Qty

Primary Key: (OrderID, ProdID)

Summary

Functional Dependencies

- $\text{OrderID} \rightarrow \text{CustID}$
- $\text{CustID} \rightarrow \text{CustName}$
- $\text{ProdID} \rightarrow \text{ProdName}, \text{Price}$
- $(\text{OrderID}, \text{ProdID}) \rightarrow \text{Qty}$

3NF Relations

1. **Customer**(CustID, CustName)
2. **Product**(ProdID, ProdName, Price)
3. **Order**(OrderID, CustID)
4. **OrderItem**(OrderID, ProdID, Qty)

This decomposition eliminates redundancy, removes update anomalies, and preserves all functional dependencies while satisfying **Third Normal Form (3NF)**.

2. A hospital maintains Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName). Identify anomalies and normalize to BCNF.

```
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE HospitalDB;
Query OK, 1 row affected (0.06 sec)

mysql> USE HospitalDB;
Database changed
mysql> CREATE TABLE Doctor ( DocID VARCHAR(10) PRIMARY KEY, DocName VARCHAR(50) NOT NULL, Specialization VARCHAR(50) NOT NULL );
Query OK, 0 rows affected (0.06 sec)

mysql> CREATE TABLE Ward ( WardNo VARCHAR(10) PRIMARY KEY, WardName VARCHAR(50) NOT NULL );
Query OK, 0 rows affected (0.11 sec)

mysql> CREATE TABLE Patient ( PID VARCHAR(10) PRIMARY KEY, PName VARCHAR(50) NOT NULL, DocID VARCHAR(10), WardNo VARCHAR(10), FOREIGN KEY (DocID) REFERENCES
  Doctor(DocID), FOREIGN KEY (WardNo) REFERENCES Ward(WardNo) );
Query OK, 0 rows affected (0.16 sec)

mysql> INSERT INTO Doctor VALUES ('D01', 'Dr. Sonu', 'Cardiology'), ('D02', 'Dr. Pranathi', 'Neurology'), ('D03', 'Dr. Roopa', 'Orthopedics');
Query OK, 3 rows affected (0.03 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Ward VALUES ('W01', 'Cardiology Ward'), ('W02', 'Neurology Ward'), ('W03', 'Orthopedic Ward');
Query OK, 3 rows affected (0.06 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Patient VALUES ('P01', 'Abhighna', 'D01', 'W01'), ('P02', 'Aishu', 'D01', 'W01'), ('P03', 'Krishna', 'D02', 'W02'), ('P04', 'Sunitha', 'D
02', 'W02');
Query OK, 4 rows affected (0.04 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> select*from doctor;
+-----+-----+-----+
| DocID | DocName | Specialization |
+-----+-----+-----+
| D01   | Dr. Sonu | Cardiology     |
| D02   | Dr. Pranathi | Neurology     |
| D03   | Dr. Roopa | Orthopedics    |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> select*from doctor;
+-----+-----+-----+
| DocID | DocName | Specialization |
+-----+-----+-----+
| D01   | Dr. Sonu | Cardiology     |
| D02   | Dr. Pranathi | Neurology     |
| D03   | Dr. Roopa | Orthopedics    |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> select*from ward;
+-----+-----+
| WardNo | WardName |
+-----+-----+
| W01    | Cardiology Ward |
| W02    | Neurology Ward |
| W03    | Orthopedic Ward |
+-----+-----+
3 rows in set (0.00 sec)

mysql> select*from patient;
+-----+-----+-----+-----+
| PID | PName | DocID | WardNo |
+-----+-----+-----+-----+
| P01 | Abhighna | D01 | W01 |
| P02 | Aishu | D01 | W01 |
| P03 | Krishna | D02 | W02 |
| P04 | Sunitha | D02 | W02 |
+-----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql> SELECT p.PID, p.PName, d.DocID, d.DocName, d.Specialization, w.WardNo, w.WardName FROM Patient p JOIN Doctor d ON p.DocID = d.DocID JOIN Ward w ON p.
WardNo = w.WardNo;
+-----+-----+-----+-----+-----+-----+-----+
| PID | PName | DocID | DocName | Specialization | WardNo | WardName |
+-----+-----+-----+-----+-----+-----+-----+
| P01 | Abhighna | D01 | Dr. Sonu | Cardiology | W01 | Cardiology Ward |
| P02 | Aishu | D01 | Dr. Sonu | Cardiology | W01 | Cardiology Ward |
| P03 | Krishna | D02 | Dr. Pranathi | Neurology | W02 | Neurology Ward |
| P04 | Sunitha | D02 | Dr. Pranathi | Neurology | W02 | Neurology Ward |
+-----+-----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

Given relation:

Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName)

Step 1: Identify Functional Dependencies

Assume:

1. $PID \rightarrow PName, DocID, WardNo$
2. $DocID \rightarrow DocName, Specialization$
3. $WardNo \rightarrow WardName$

Therefore, the candidate key is:

PID

Because:

$PID \rightarrow DocID \rightarrow DocName, Specialization$

and

$PID \rightarrow WardNo \rightarrow WardName$

Identify Anomalies

1. Update Anomaly

If a doctor's specialization changes, the same information may need to be updated for many patient records.

Example:

$D101 \rightarrow Cardiology$

If Dr. D101 has 20 patients, the specialization may be repeated 20 times.

2. Insertion Anomaly

A new doctor cannot be added unless the doctor is assigned to a patient.

Similarly, a new ward cannot be stored unless at least one patient is assigned to it.

3. Deletion Anomaly

If the only patient assigned to a doctor is deleted, information about that doctor may also be lost.

Check BCNF

A relation is in **BCNF** if:

For every non-trivial FD $X \rightarrow Y$, X must be a superkey.

Consider:

FD 1

$PID \rightarrow PName, DocID, WardNo$

PID is the candidate key, so **BCNF satisfied**.

FD 2

$DocID \rightarrow DocName, Specialization$

DocID is **not** a candidate key of the original relation.

Therefore, **BCNF violated**.

FD 3

$\text{WardNo} \rightarrow \text{WardName}$

WardNo is **not** a candidate key of the original relation.

Therefore, **BCNF violated**.

Decompose into BCNF

Relation 1: Patient

Patient(PID, PName, DocID, WardNo)

Primary Key: PID

FD:

$\text{PID} \rightarrow \text{PName}, \text{DocID}, \text{WardNo}$

This relation is in BCNF because PID is a superkey.

Relation 2: Doctor

Doctor(DocID, DocName, Specialization)

Primary Key: DocID

FD:

$\text{DocID} \rightarrow \text{DocName}, \text{Specialization}$

DocID is the key, so this relation is in BCNF.

Relation 3: Ward

Ward(WardNo, WardName)

Primary Key: WardNo

FD:

$\text{WardNo} \rightarrow \text{WardName}$

WardNo is the key, so this relation is in BCNF.

3. A university stores Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade). Analyze FDs and apply 2NF decomposition.

```
mysql> CREATE DATABASE UniversityDB;
Query OK, 1 row affected (0.03 sec)

mysql> USE UniversityDB;
Database changed
mysql> CREATE TABLE Student ( SID VARCHAR(10) PRIMARY KEY, SName VARCHAR(50) NOT NULL );
Query OK, 0 rows affected (0.14 sec)

mysql> CREATE TABLE Course ( CourseID VARCHAR(10) PRIMARY KEY, CourseName VARCHAR(50) NOT NULL, Faculty VARCHAR(50) NOT NULL );
Query OK, 0 rows affected (0.15 sec)

mysql> CREATE TABLE Enrollment ( SID VARCHAR(10), CourseID VARCHAR(10), Grade VARCHAR(5), PRIMARY KEY (SID, CourseID), FOREIGN KEY (SID) REFERENCES Student(SID), FOREIGN KEY (CourseID) REFERENCES Course(CourseID) );
Query OK, 0 rows affected (0.18 sec)

mysql> INSERT INTO Student VALUES ('S01', 'ravuru'), ('S02', 'aishu'), ('S03', 'krishna');
Query OK, 3 rows affected (0.04 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Course VALUES ('C01', 'DBMS', 'Dr. Kiran'), ('C02', 'Operating Systems', 'Dr. Pranathi'), ('C03', 'Python', 'Dr. Ramesh');
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Enrollment VALUES ('S01', 'C01', 'A'), ('S01', 'C02', 'B+'), ('S02', 'C01', 'A+'), ('S03', 'C03', 'A');
Query OK, 4 rows affected (0.06 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM Student;
+----+-----+
| SID | SName |
+----+-----+
| S01 | ravuru |
| S02 | aishu  |
| S03 | krishna |
+----+-----+
3 rows in set (0.00 sec)

mysql> SELECT * FROM Course;
+----+-----+-----+
| CourseID | CourseName | Faculty |
+----+-----+-----+
| C01      | DBMS       | Dr. Kiran |
| C02      | Operating Systems | Dr. Pranathi |
| C03      | Python     | Dr. Ramesh |
+----+-----+-----+
```

```
mysql> SELECT * FROM Course;
+----+-----+-----+
| CourseID | CourseName | Faculty |
+----+-----+-----+
| C01      | DBMS       | Dr. Kiran |
| C02      | Operating Systems | Dr. Pranathi |
| C03      | Python     | Dr. Ramesh |
+----+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT * FROM Enrollment;
+----+-----+-----+
| SID | CourseID | Grade |
+----+-----+-----+
| S01 | C01      | A      |
| S01 | C02      | B+     |
| S02 | C01      | A+     |
| S03 | C03      | A      |
+----+-----+-----+
4 rows in set (0.00 sec)

mysql> SELECT e.SID, s.SName, e.CourseID, c.CourseName, c.Faculty, e.Grade FROM Enrollment e JOIN Student s ON e.SID = s.SID JOIN Course c ON e.CourseID = c.CourseID;
+----+-----+-----+-----+-----+-----+
| SID | SName | CourseID | CourseName | Faculty | Grade |
+----+-----+-----+-----+-----+-----+
| S01 | ravuru | C01      | DBMS       | Dr. Kiran | A      |
| S01 | ravuru | C02      | Operating Systems | Dr. Pranathi | B+     |
| S02 | aishu  | C01      | DBMS       | Dr. Kiran | A+     |
| S03 | krishna | C03      | Python     | Dr. Ramesh | A      |
+----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

Given relation:

Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade)

Step 1: Identify Functional Dependencies

Assume:

1. $SID \rightarrow SName$
2. $CourseID \rightarrow CourseName, Faculty$
3. $(SID, CourseID) \rightarrow Grade$

Step 2: Find Candidate Key

A student can enroll in multiple courses, and a course can have multiple students.

Therefore:

Candidate Key = (SID, CourseID)

Step 3: Check 1NF

The relation is in **1NF** because all attributes contain atomic values.

Step 4: Check 2NF

For 2NF, there should be **no partial dependency** on a part of the composite key.

We have:

$SID \rightarrow SName$

Here, SName depends only on SID, not on the complete (SID, CourseID) key.

Also:

$CourseID \rightarrow CourseName, Faculty$

Here, CourseName and Faculty depend only on CourseID.

Therefore, the original relation **is not in 2NF**.

Conclusion

The original **Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade)** relation has the composite candidate key **(SID, CourseID)**. Since SName depends only on SID, and CourseName and Faculty depend only on CourseID, the relation contains partial dependencies and is not in 2NF. By decomposing it into **Student(SID, SName)**, **Course(CourseID, CourseName, Faculty)**, and **Enrollment(SID, CourseID, Grade)**, all partial dependencies are eliminated. The resulting relations satisfy **Second Normal Form (2NF)** and reduce data redundancy and update, insertion, and deletion anomalies.

4. Given the airline booking case: Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date). Normalize up to BCNF.

```
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE AirlineDB;
Query OK, 1 row affected (0.06 sec)

mysql> USE AirlineDB;
Database changed
mysql> CREATE TABLE Passenger ( PassID VARCHAR(10) PRIMARY KEY, PassName VARCHAR(50) NOT NULL );
Query OK, 0 rows affected (0.10 sec)

mysql> CREATE TABLE Flight ( FlightNo VARCHAR(10) PRIMARY KEY, DeptCity VARCHAR(50), ArrCity VARCHAR(50) );
Query OK, 0 rows affected (0.10 sec)

mysql> CREATE TABLE Booking ( FlightNo VARCHAR(10), PassID VARCHAR(10), Seat VARCHAR(10), Date DATE, PRIMARY KEY (FlightNo, PassID), FOREIGN KEY (FlightNo)
REFERENCES Flight(FlightNo), FOREIGN KEY (PassID) REFERENCES Passenger(PassID) );
Query OK, 0 rows affected (0.20 sec)

mysql> INSERT INTO Passenger VALUES ('P01', 'Ramesh'), ('P02', 'Aishu'), ('P03', 'Pranathi');
Query OK, 3 rows affected (0.05 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Flight VALUES ('F101', 'Chennai', 'Delhi'), ('F102', 'Mumbai', 'Bangalore');
Query OK, 2 rows affected (0.07 sec)
Records: 2 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Booking VALUES ('F101', 'P01', '12A', '2026-08-15'), ('F101', 'P02', '12B', '2026-08-15'), ('F102', 'P03', '15C', '2026-08-16');
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> SELECT b.FlightNo, p.PassID, p.PassName, b.Seat, f.DeptCity, f.ArrCity, b.Date FROM Booking b JOIN Passenger p ON b.PassID = p.PassID JOIN Flight f ON
b.FlightNo = f.FlightNo;
+-----+-----+-----+-----+-----+-----+-----+
| FlightNo | PassID | PassName | Seat | DeptCity | ArrCity | Date |
+-----+-----+-----+-----+-----+-----+-----+
| F101    | P01    | Ramesh   | 12A  | Chennai  | Delhi   | 2026-08-15 |
| F101    | P02    | Aishu    | 12B  | Chennai  | Delhi   | 2026-08-15 |
| F102    | P03    | Pranathi | 15C  | Mumbai   | Bangalore | 2026-08-16 |
+-----+-----+-----+-----+-----+-----+-----+
3 rows in set (0.05 sec)
```

Given Relation

Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date)

We need to identify the **functional dependencies**, find the **candidate key**, and normalize the relation up to **BCNF**.

1. Functional Dependencies

Assume:

- Each passenger ID uniquely identifies a passenger.
- Each flight number identifies its departure and arrival cities.
- A passenger can book a particular flight and gets one seat.
- A flight operates on a particular date.

The functional dependencies are:

1. **PassID** → **PassName**
2. **FlightNo** → **DeptCity, ArrCity**
3. **(FlightNo, PassID)** → **Seat, Date**

Therefore:

FDs = { PassID → PassName, FlightNo → DeptCity, ArrCity, (FlightNo, PassID) → Seat, Date }

2. Candidate Key

PassID alone cannot identify a booking because one passenger can book multiple flights.

FlightNo alone cannot identify a booking because many passengers can book the same flight.

Therefore:

Candidate Key = (FlightNo, PassID)

Its closure is:

(FlightNo, PassID)+

→ FlightNo, PassID
→ DeptCity, ArrCity
→ PassName
→ Seat, Date

So it determines all attributes.

3. Check 1NF

The relation is in **1NF** because:

- All attributes contain atomic values.
- There are no repeating groups.
- Each booking is represented as a separate record.

Therefore:

Booking is in 1NF.

4. Check 2NF

The primary key is:

(FlightNo, PassID)

Now identify partial dependencies.

Partial Dependency 1

PassID → PassName

PassName depends only on PassID, which is part of the composite key.

Partial Dependency 2

FlightNo → DeptCity, ArrCity

DeptCity and ArrCity depend only on FlightNo, which is also part of the composite key.

Therefore, the relation is **not in 2NF**.

5. Decompose into 2NF

Remove the partial dependencies.

Passenger

Passenger(PassID, PassName)

FD:

PassID $\rightarrow$ PassName

Flight

Flight(FlightNo, DeptCity, ArrCity)

FD:

FlightNo $\rightarrow$ DeptCity, ArrCity

Booking

Booking(FlightNo, PassID, Seat, Date)

FD:

(FlightNo, PassID) $\rightarrow$ Seat, Date

Now there are no partial dependencies.

6. Check 3NF

Passenger

PassID $\rightarrow$ PassName

PassID is the key.

Therefore, it satisfies 3NF.

Flight

FlightNo $\rightarrow$ DeptCity, ArrCity

FlightNo is the key.

Therefore, it satisfies 3NF.

Booking

(FlightNo, PassID) $\rightarrow$ Seat, Date

(FlightNo, PassID) is the key.

Therefore, it satisfies 3NF.

7. Check BCNF

A relation is in **BCNF** if, for every non-trivial functional dependency:

The determinant must be a superkey.

Passenger

$\text{PassID} \rightarrow \text{PassName}$

PassID is a superkey.

BCNF satisfied.

Flight

$\text{FlightNo} \rightarrow \text{DeptCity}, \text{ArrCity}$

FlightNo is a superkey.

BCNF satisfied.

Booking

$(\text{FlightNo}, \text{PassID}) \rightarrow \text{Seat}, \text{Date}$

$(\text{FlightNo}, \text{PassID})$ is a superkey.

BCNF satisfied.

Therefore, all three relations are in **BCNF**.

Final BCNF Decomposition

Passenger(PassID, PassName)

Flight(FlightNo, DeptCity, ArrCity)

Booking(FlightNo, PassID, Seat, Date)

Final Result

Booking

$(\text{FlightNo}, \text{PassID}, \text{PassName}, \text{Seat}, \text{DeptCity}, \text{ArrCity}, \text{Date})$

↓

Passenger

$(\text{PassID}, \text{PassName})$

Flight

$(\text{FlightNo}, \text{DeptCity}, \text{ArrCity})$

Booking

$(\text{FlightNo}, \text{PassID}, \text{Seat}, \text{Date})$

Conclusion

The original Booking relation contains partial dependencies because passenger details depend only on PassID and flight details depend only on FlightNo. Decomposing it into **Passenger, Flight, and Booking** removes these dependencies. In each resulting relation, every determinant is a superkey, so the final decomposition is in **BCNF**.

5. Analyze an HR system schema for a multinational company. Identify all functional and transitive dependencies and normalize to 3NF.

```
mysql> CREATE TABLE Employee ( EmpID VARCHAR(10) PRIMARY KEY, EmpName VARCHAR(50) NOT NULL, DeptID VARCHAR(10), ManagerID VARCHAR(10), FOREIGN KEY (DeptID)
REFERENCES Department(DeptID));
Query OK, 0 rows affected (0.16 sec)

mysql> INSERT INTO Department VALUES ('D01', 'IT'), ('D02', 'HR'), ('D03', 'Finance');
Query OK, 3 rows affected (0.85 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Employee VALUES ('E01', 'Ramesh', 'D01', 'E05'), ('E02', 'Aishu', 'D02', 'E06'), ('E03', 'Sunitha', 'D03', 'E07');
Query OK, 3 rows affected (0.86 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM Department;
+-----+-----+
| DeptID | DeptName |
+-----+-----+
| D01    | IT       |
| D02    | HR       |
| D03    | Finance  |
+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT * FROM Employee;
+-----+-----+-----+-----+
| EmpID | EmpName | DeptID | ManagerID |
+-----+-----+-----+-----+
| E01   | Ramesh  | D01    | E05       |
| E02   | Aishu   | D02    | E06       |
| E03   | Sunitha | D03    | E07       |
+-----+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT e.EmpID, e.EmpName, e.DeptID, d.DeptName, e.ManagerID FROM Employee e JOIN Department d ON e.DeptID = d.DeptID;
+-----+-----+-----+-----+-----+
| EmpID | EmpName | DeptID | DeptName | ManagerID |
+-----+-----+-----+-----+-----+
| E01   | Ramesh  | D01    | IT       | E05       |
| E02   | Aishu   | D02    | HR       | E06       |
| E03   | Sunitha | D03    | Finance  | E07       |
+-----+-----+-----+-----+-----+
3 rows in set (0.05 sec)
```

Given Relation

Assume the HR system has the following relation:

Employee(EmpID, EmpName, DeptID, DeptName, ManagerID, ManagerName, LocationID, LocationName)

1. Identify Functional Dependencies

The functional dependencies are:

1. **EmpID $\rightarrow$ EmpName, DeptID, ManagerID**
2. **DeptID $\rightarrow$ DeptName, LocationID**
3. **ManagerID $\rightarrow$ ManagerName**
4. **LocationID $\rightarrow$ LocationName**

2. Find the Candidate Key

EmpID uniquely identifies an employee.

Therefore:

Candidate Key = EmpID

3. Identify Transitive Dependencies

A transitive dependency occurs when a non-key attribute depends on another non-key attribute.

Dependency 1

EmpID $\rightarrow$ DeptID

and

DeptID $\rightarrow$ DeptName

Therefore:

EmpID $\rightarrow$ DeptName

This is a **transitive dependency**.

Dependency 2

EmpID $\rightarrow$ ManagerID

and

ManagerID $\rightarrow$ ManagerName

Therefore:

EmpID $\rightarrow$ ManagerName

This is a **transitive dependency**.

Dependency 3

EmpID $\rightarrow$ DeptID

DeptID $\rightarrow$ LocationID

LocationID $\rightarrow$ LocationName

Therefore:

EmpID $\rightarrow$ LocationName

This is also a **transitive dependency**.

4. Check 1NF

The relation is in **1NF** because:

- Each attribute contains atomic values.
- There are no repeating groups.
- Each employee is represented by a separate record.

5. Check 2NF

The candidate key is a single attribute:

EmpID

Since there is no composite key, partial dependency cannot occur.

Therefore, the relation is already in **2NF**.

6. Check 3NF

The relation is **not in 3NF** because of the transitive dependencies:

- $\text{EmpID} \rightarrow \text{DeptID} \rightarrow \text{DeptName}$
- $\text{EmpID} \rightarrow \text{ManagerID} \rightarrow \text{ManagerName}$
- $\text{EmpID} \rightarrow \text{DeptID} \rightarrow \text{LocationID} \rightarrow \text{LocationName}$

Therefore, we need to decompose the relation.

7. 3NF Decomposition

Employee

Employee(EmpID, EmpName, DeptID, ManagerID)

FD:

$\text{EmpID} \rightarrow \text{EmpName, DeptID, ManagerID}$

Department

Department(DeptID, DeptName, LocationID)

FD:

$\text{DeptID} \rightarrow \text{DeptName, LocationID}$

Manager

Manager(ManagerID, ManagerName)

FD:

$\text{ManagerID} \rightarrow \text{ManagerName}$

Location

Location(LocationID, LocationName)

FD:

$\text{LocationID} \rightarrow \text{LocationName}$

8. Final 3NF Schema

Employee

EmpID (PK)

EmpName

DeptID (FK)

ManagerID (FK)

Department

DeptID (PK)

DeptName

LocationID (FK)

Manager

ManagerID (PK)

ManagerName\

Location

LocationID (PK)

LocationName

9. Dependency Analysis

Functional Dependencies

EmpID $\rightarrow$ EmpName, DeptID, ManagerID

DeptID $\rightarrow$ DeptName, LocationID

ManagerID $\rightarrow$ ManagerName

LocationID $\rightarrow$ LocationName

Transitive Dependencies

EmpID $\rightarrow$ DeptID $\rightarrow$ DeptName

EmpID $\rightarrow$ ManagerID $\rightarrow$ ManagerName

EmpID $\rightarrow$ DeptID $\rightarrow$ LocationID $\rightarrow$ LocationName

Conclusion

The original HR relation is in **2NF** but not in **3NF** because department, manager, and location information has transitive dependencies on EmpID. By decomposing the relation into **Employee, Department, Manager, and Location**, all transitive dependencies are removed. The resulting relations satisfy **3NF**, reducing data redundancy and preventing update, insertion, and deletion anomalies.

6. Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate). Identify MVDs and decompose to 4NF.

```
mysql> CREATE DATABASE LibraryDB;
Query OK, 1 row affected (0.06 sec)

mysql> USE LibraryDB;
Database changed
mysql> CREATE TABLE Member ( MemberID VARCHAR(10) PRIMARY KEY, MemberName VARCHAR(50) NOT NULL );
Query OK, 0 rows affected (0.06 sec)

mysql> CREATE TABLE Book ( BookID VARCHAR(10) PRIMARY KEY, Title VARCHAR(100) NOT NULL, Author VARCHAR(50), Genre VARCHAR(50) );
Query OK, 0 rows affected (0.08 sec)

mysql> CREATE TABLE Borrowing ( MemberID VARCHAR(10), BookID VARCHAR(10), BorrowDate DATE, PRIMARY KEY (MemberID, BookID), FOREIGN KEY (MemberID) REFERENCES Member(MemberID), FOREIGN KEY (BookID) REFERENCES Book(BookID) );
Query OK, 0 rows affected (0.10 sec)

mysql> INSERT INTO Member VALUES ('M01', 'Ramesh'), ('M02', 'Sunitha'), ('M03', 'Lehan');
Query OK, 3 rows affected (0.06 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Book VALUES ('B01', 'DBMS', 'Kumar', 'Education'), ('B02', 'Operating Systems', 'Galvin', 'Education'), ('B03', 'Python', 'Mark Lutz', 'Programming');
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Borrowing VALUES ('M01', 'B01', '2026-08-01'), ('M01', 'B02', '2026-08-03'), ('M02', 'B03', '2026-08-05');
Query OK, 3 rows affected (0.07 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> SELECT m.MemberID, m.MemberName, b.BookID, b.Title, b.Author, b.Genre, br.BorrowDate FROM Borrowing br JOIN Member m ON br.MemberID = m.MemberID JOIN Book b ON br.BookID = b.BookID;
+-----+-----+-----+-----+-----+-----+-----+
| MemberID | MemberName | BookID | Title | Author | Genre | BorrowDate |
+-----+-----+-----+-----+-----+-----+-----+
| M01      | Ramesh     | B01    | DBMS  | Kumar  | Education | 2026-08-01 |
| M01      | Ramesh     | B02    | Operating Systems | Galvin | Education | 2026-08-03 |
| M02      | Sunitha    | B03    | Python | Mark Lutz | Programming | 2026-08-05 |
+-----+-----+-----+-----+-----+-----+-----+
3 rows in set (0.05 sec)
```

Given Relation

Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate)

The question asks us to **identify multivalued dependencies (MVDs)** and decompose the relation into **4NF**.

1. Identify Functional Dependencies

First, identify the normal functional dependencies:

- **MemberID** → **MemberName**
- **BookID** → **Title, Author, Genre**

This means:

- A MemberID determines the member's name.
- A BookID determines the book's title, author, and genre.

2. Identify Multivalued Dependencies

Assume that a member can:

- Borrow multiple books.
- Have multiple independent borrowing records.

Therefore:

MemberID →→ BookID

This means a member can be associated with multiple books independently.

Also:

MemberID $\twoheadrightarrow$ BorrowDate

A member can have multiple borrowing dates independently of the books.

Thus, the important MVD is:

MemberID $\twoheadrightarrow$ BookID

and the corresponding independent multivalued relationship is:

MemberID $\twoheadrightarrow$ BorrowDate

3. Why 4NF is Violated

A relation is in **4NF** if for every non-trivial MVD:

$X \twoheadrightarrow Y$, X must be a superkey.

Here:

MemberID $\twoheadrightarrow$ BookID

But MemberID is **not a superkey** of the original relation because it does not determine a single book.

Therefore, the relation **violates 4NF**.

4. Decompose the Relation

Separate the independent multivalued facts.

Relation 1: Member

Member(MemberID, MemberName)

FD:

MemberID $\rightarrow$ MemberName

Relation 2: Book

Book(BookID, Title, Author, Genre)

FD:

BookID $\rightarrow$ Title, Author, Genre

Relation 3: Borrowing

Borrowing(MemberID, BookID, BorrowDate)

This relation records which member borrowed which book and when.

5. Final 4NF Schema

Member

MemberID (PK)

MemberName

Book

BookID (PK)

Title

Author

Genre

Borrowing

MemberID (FK)

BookID (FK)

BorrowDate

6. Result

The original relation:

Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate)

is decomposed into:

Member(MemberID, MemberName)

Book(BookID, Title, Author, Genre)

Borrowing(MemberID, BookID, BorrowDate)

The decomposition removes the **multivalued dependency** and eliminates unnecessary combinations of members, books, and borrowing dates.

Final Answer

MVD:

MemberID $\twoheadrightarrow$ BookID

Other independent relationship:

MemberID $\twoheadrightarrow$ BorrowDate

4NF Decomposition:

Member(MemberID, MemberName)

Book(BookID, Title, Author, Genre)

Borrowing(MemberID, BookID, BorrowDate)

Thus, the library schema is decomposed into **4NF** by separating the independent multivalued facts about members, books, and borrowing records.

7. Analyze a telecom billing schema and identify all candidate keys, primary keys, and functional dependencies before normalization

```
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE TelecomDB;
Query OK, 1 row affected (0.05 sec)

mysql> USE TelecomDB;
Database changed
mysql> CREATE TABLE Telecom ( CustomerID VARCHAR(10), CustomerName VARCHAR(50), PhoneNo VARCHAR(15), BillID VARCHAR(10), BillDate DATE, Amount DECIMAL(10,2)
, PRIMARY KEY (BillID), UNIQUE (PhoneNo), UNIQUE (CustomerID) );
Query OK, 0 rows affected (0.17 sec)

mysql> INSERT INTO Telecom VALUES ('C01', 'Sonu', '9876543210', 'B01', '2026-08-01', 599.00), ('C02', 'Pranathi', '9876543211', 'B02', '2026-08-02', 799.00)
, ('C03', 'Roopa', '9876543212', 'B03', '2026-08-03', 499.00);
Query OK, 3 rows affected (0.20 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM Telecom;
+-----+-----+-----+-----+-----+-----+
| CustomerID | CustomerName | PhoneNo | BillID | BillDate | Amount |
+-----+-----+-----+-----+-----+-----+
| C01       | Sonu         | 9876543210 | B01    | 2026-08-01 | 599.00 |
| C02       | Pranathi     | 9876543211 | B02    | 2026-08-02 | 799.00 |
| C03       | Roopa        | 9876543212 | B03    | 2026-08-03 | 499.00 |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.04 sec)

mysql> SELECT CustomerID, COUNT(*) FROM Telecom GROUP BY CustomerID;
+-----+-----+
| CustomerID | COUNT(*) |
+-----+-----+
| C01       | 1        |
| C02       | 1        |
| C03       | 1        |
+-----+-----+
3 rows in set (0.01 sec)

mysql> SELECT PhoneNo, COUNT(*) FROM Telecom GROUP BY PhoneNo;
+-----+-----+
| PhoneNo | COUNT(*) |
+-----+-----+
| 9876543210 | 1        |
| 9876543211 | 1        |
| 9876543212 | 1        |
+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT BillID, COUNT(*) FROM Telecom GROUP BY BillID;
mysql> SELECT BillID, COUNT(*) FROM Telecom GROUP BY BillID;
+-----+-----+
| BillID | COUNT(*) |
+-----+-----+
| B01    | 1        |
| B02    | 1        |
| B03    | 1        |
+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT CustomerID, CustomerName, PhoneNo, BillID, BillDate, Amount FROM Telecom;
+-----+-----+-----+-----+-----+-----+
| CustomerID | CustomerName | PhoneNo | BillID | BillDate | Amount |
+-----+-----+-----+-----+-----+-----+
| C01       | Sonu         | 9876543210 | B01    | 2026-08-01 | 599.00 |
| C02       | Pranathi     | 9876543211 | B02    | 2026-08-02 | 799.00 |
| C03       | Roopa        | 9876543212 | B03    | 2026-08-03 | 499.00 |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

Since the question does not provide a specific schema, we can use a typical telecom billing relation:

Billing(BillID, CustomerID, CustomerName, PhoneNo, PlanID, PlanName, MonthlyCharge, CallID, CallDate, CallDuration, CallCharge)

1. Understand the Attributes

- **BillID** – uniquely identifies a bill.
- **CustomerID** – uniquely identifies a customer.
- **CustomerName** – name of the customer.
- **PhoneNo** – customer's phone number.
- **PlanID** – identifies the telecom plan.
- **PlanName** – name of the plan.
- **MonthlyCharge** – monthly charge of the plan.
- **CallID** – uniquely identifies a call.
- **CallDate** – date of the call.
- **CallDuration** – duration of the call.
- **CallCharge** – charge for the call.

2. Functional Dependencies

The important functional dependencies are:

Customer Dependency

CustomerID $\rightarrow$ CustomerName, PhoneNo, PlanID

A customer ID determines the customer's details and subscribed plan.

Plan Dependency

PlanID $\rightarrow$ PlanName, MonthlyCharge

A plan ID determines the plan name and monthly charge.

Bill Dependency

BillID $\rightarrow$ CustomerID

A bill ID determines the customer associated with the bill.

Call Dependency

CallID $\rightarrow$ CustomerID, CallDate, CallDuration, CallCharge

A call ID uniquely identifies a call and its details.

Therefore, the main FD set is:

F = {

- CustomerID $\rightarrow$ CustomerName, PhoneNo, PlanID
- PlanID $\rightarrow$ PlanName, MonthlyCharge
- BillID $\rightarrow$ CustomerID
- CallID $\rightarrow$ CustomerID, CallDate, CallDuration, CallCharge

}

3. Candidate Keys

A **candidate key** is a minimal set of attributes that uniquely identifies every tuple.

Candidate Key 1

BillID

Because:

BillID $\rightarrow$ CustomerID

and

CustomerID $\rightarrow$ CustomerName, PhoneNo, PlanID

and

$\text{PlanID} \rightarrow \text{PlanName, MonthlyCharge}$

However, if the relation contains multiple call records for the same bill, BillID alone will not identify each row.

Therefore, under this schema, the key would need to include the call identifier.

Candidate Key

(BillID, CallID)

This combination uniquely identifies a billing record.

4. Check Candidate Key Closure

Take:

(BillID, CallID)+

Starting with:

BillID, CallID

From:

$\text{BillID} \rightarrow \text{CustomerID}$

we get:

CustomerID

From:

$\text{CustomerID} \rightarrow \text{CustomerName, PhoneNo, PlanID}$

we get:

CustomerName, PhoneNo, PlanID

From:

$\text{PlanID} \rightarrow \text{PlanName, MonthlyCharge}$

we get:

PlanName, MonthlyCharge

From:

$\text{CallID} \rightarrow \text{CustomerID, CallDate, CallDuration, CallCharge}$

we get:

CallDate, CallDuration, CallCharge

Therefore:

(BillID, CallID)+ = all attributes

So (BillID, CallID) is a candidate key.

5. Primary Key

A primary key is selected from the candidate keys.

Therefore:

Primary Key = (BillID, CallID)

6. Other Candidate Key Possibility

If CallID is globally unique and determines the entire billing record, then:

CallID → all call-related attributes

But CallID alone does not determine BillID unless that dependency is explicitly given.

Therefore, based on the stated FDs, we should **not assume CallID alone is a candidate key**.

7. Transitive Dependencies

There are also transitive dependencies.

Example 1

CustomerID → PlanID

and

PlanID → PlanName, MonthlyCharge

Therefore:

CustomerID → PlanName, MonthlyCharge

This is a transitive dependency.

Example 2

BillID → CustomerID

and

CustomerID → CustomerName, PhoneNo, PlanID

Therefore:

BillID $\rightarrow$ CustomerName, PhoneNo, PlanID

This is also transitive.

8. Final Analysis Candidate Key

(BillID, CallID)

Primary Key

(BillID, CallID)

Functional Dependencies

- CustomerID $\rightarrow$ CustomerName, PhoneNo, PlanID
- PlanID $\rightarrow$ PlanName, MonthlyCharge
- BillID $\rightarrow$ CustomerID
- CallID $\rightarrow$ CustomerID, CallDate, CallDuration, CallCharge

Important Transitive Dependencies

- CustomerID $\rightarrow$ PlanID $\rightarrow$ PlanName, MonthlyCharge
- BillID $\rightarrow$ CustomerID $\rightarrow$ CustomerName, PhoneNo
- BillID $\rightarrow$ CustomerID $\rightarrow$ PlanID $\rightarrow$ PlanName, MonthlyCharge

Conclusion

The telecom billing schema contains several functional and transitive dependencies. The candidate key must uniquely identify each billing record, which under the stated assumptions is **(BillID, CallID)**. This analysis provides the basis for further normalization into **2NF, 3NF, or BCNF** by separating customer, plan, bill, and call information.

8. A retail inventory system has Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse). Apply complete normalization.

```
mysql> CREATE TABLE Product ( PID VARCHAR(10) PRIMARY KEY, PName VARCHAR(50) NOT NULL, SupplierID VARCHAR(10), Category VARCHAR(50), Price DECIMAL(10,2), Warehouse VARCHAR(50), FOREIGN KEY (SupplierID) REFERENCES Supplier(SupplierID) );
Query OK, 0 rows affected (0.14 sec)

mysql> INSERT INTO Supplier VALUES ('S01', 'ABC Suppliers'), ('S02', 'XYZ Traders'), ('S03', 'Global Supplies');
Query OK, 3 rows affected (0.05 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Product VALUES ('P01', 'Laptop', 'S01', 'Electronics', 50000, 'W01'), ('P02', 'Mouse', 'S01', 'Accessories', 1000, 'W01'), ('P03', 'Keyboard', 'S02', 'Accessories', 2000, 'W02'), ('P04', 'Monitor', 'S03', 'Electronics', 15000, 'W02');
Query OK, 4 rows affected (0.02 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM Product;
+----+-----+-----+-----+-----+-----+
| PID | PName | SupplierID | Category | Price | Warehouse |
+----+-----+-----+-----+-----+-----+
| P01 | Laptop | S01 | Electronics | 50000.00 | W01 |
| P02 | Mouse | S01 | Accessories | 1000.00 | W01 |
| P03 | Keyboard | S02 | Accessories | 2000.00 | W02 |
| P04 | Monitor | S03 | Electronics | 15000.00 | W02 |
+----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql> SELECT * FROM Supplier;
+-----+-----+
| SupplierID | SupplierName |
+-----+-----+
| S01 | ABC Suppliers |
| S02 | XYZ Traders |
| S03 | Global Supplies |
+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT p.PID, p.PName, s.SupplierID, s.SupplierName, p.Category, p.Price, p.Warehouse FROM Product p JOIN Supplier s ON p.SupplierID = s.SupplierID;
+----+-----+-----+-----+-----+-----+
| PID | PName | SupplierID | SupplierName | Category | Price | Warehouse |
+----+-----+-----+-----+-----+-----+
| P01 | Laptop | S01 | ABC Suppliers | Electronics | 50000.00 | W01 |
| P02 | Mouse | S01 | ABC Suppliers | Accessories | 1000.00 | W01 |
| P03 | Keyboard | S02 | XYZ Traders | Accessories | 2000.00 | W02 |
| P04 | Monitor | S03 | Global Supplies | Electronics | 15000.00 | W02 |
+----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

Given relation:

Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse)

We need to identify the functional dependencies and normalize the relation completely up to **3NF**.

1. Functional Dependencies

Assume the following dependencies:

- **PID → PName, SupplierID, Category, Price, Warehouse**
- **SupplierID → SupplierName**

Here, PID uniquely identifies a product, while SupplierID determines the supplier's name.

2. Candidate Key

Since PID determines all other attributes:

PID → PName, SupplierID, Category, Price, Warehouse

and

SupplierID → SupplierName

Therefore, PID determines every attribute.

Candidate Key = **PID**

3. Check 1NF

The relation is in **1NF** because:

- All attributes contain atomic values.
- There are no repeating groups.
- Each product is represented by a separate record.

So:

Product is in 1NF.

4. Check 2NF

The candidate key is a **single attribute**, PID.

Therefore, partial dependency cannot exist.

So:

Product is already in 2NF.

5. Check 3NF

Consider:

PID → SupplierID

and

SupplierID → SupplierName

Therefore:

PID → SupplierID → SupplierName

This is a **transitive dependency**.

SupplierName depends on SupplierID, rather than directly on the primary key PID.

Therefore, the original relation is **not in 3NF**.

6. Decompose into 3NF

Separate the supplier information from the product information.

Product Relation

Product(PID, PName, SupplierID, Category, Price, Warehouse)

Functional dependency:

PID → PName, SupplierID, Category, Price, Warehouse

Supplier Relation

Supplier(SupplierID, SupplierName)

Functional dependency:

SupplierID $\rightarrow$ SupplierName

7. Final Normalized Schema

Product

PID (PK)
PName
SupplierID (FK)
Category
Price
Warehouse

Supplier

SupplierID (PK)
SupplierName

8. Normalization Summary

Normal Form	Result	Reason
1NF	✓	Attributes are atomic
2NF	✓	Single-attribute key, so no partial dependency
3NF	✓ after decomposition	Transitive dependency removed
BCNF	✓	Determinants in final relations are candidate keys

Final Answer

Original:

Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse)

FDs:

- PID $\rightarrow$ PName, SupplierID, Category, Price, Warehouse
- SupplierID $\rightarrow$ SupplierName

Candidate Key:

- PID

Transitive Dependency:

- PID $\rightarrow$ SupplierID $\rightarrow$ SupplierName

Final 3NF/BCNF decomposition:

- Product(PID, PName, SupplierID, Category, Price, Warehouse)
- Supplier(SupplierID, SupplierName)

Thus, complete normalization removes the supplier-name redundancy and eliminates the associated **update, insertion, and deletion anomalies**.

9. Study the given poorly normalized schema for a School ERP and propose a fully normalized design up to BCNF with justification.

```
mysql> CREATE DATABASE SchoolERP;
Query OK, 1 row affected (0.06 sec)

mysql> USE SchoolERP;
Database changed
mysql> CREATE TABLE Teacher ( TeacherID VARCHAR(10) PRIMARY KEY, TeacherName VARCHAR(50) NOT NULL );
Query OK, 0 rows affected (0.09 sec)

mysql> CREATE TABLE Class ( ClassID VARCHAR(10) PRIMARY KEY, ClassName VARCHAR(50) NOT NULL, TeacherID VARCHAR(10), FOREIGN KEY (TeacherID) REFERENCES Teacher(TeacherID) );
Query OK, 0 rows affected (0.10 sec)

mysql> CREATE TABLE Student ( StudentID VARCHAR(10) PRIMARY KEY, StudentName VARCHAR(50) NOT NULL, ClassID VARCHAR(10), FOREIGN KEY (ClassID) REFERENCES Class(ClassID) );
Query OK, 0 rows affected (0.10 sec)

mysql> CREATE TABLE Subject ( SubjectID VARCHAR(10) PRIMARY KEY, SubjectName VARCHAR(50) NOT NULL );
Query OK, 0 rows affected (0.08 sec)

mysql> CREATE TABLE Enrollment ( StudentID VARCHAR(10), SubjectID VARCHAR(10), Marks INT, PRIMARY KEY (StudentID, SubjectID), FOREIGN KEY (StudentID) REFERENCES Student(StudentID), FOREIGN KEY (SubjectID) REFERENCES Subject(SubjectID) );
Query OK, 0 rows affected (0.11 sec)

mysql> INSERT INTO Teacher VALUES ('T01', 'Sonu'), ('T02', 'Pranathi');
Query OK, 2 rows affected (0.07 sec)
Records: 2 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Class VALUES ('C01', '10-A', 'T01'), ('C02', '10-B', 'T02');
Query OK, 2 rows affected (0.06 sec)
Records: 2 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Student VALUES ('S01', 'Ramesh', 'C01'), ('S02', 'Aishu', 'C01'), ('S03', 'Lehan', 'C02');
Query OK, 3 rows affected (0.07 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Subject VALUES ('SUB01', 'DBMS'), ('SUB02', 'Mathematics'), ('SUB03', 'Python');
Query OK, 3 rows affected (0.05 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Enrollment VALUES ('S01', 'SUB01', 85), ('S01', 'SUB02', 90), ('S02', 'SUB01', 88), ('S03', 'SUB03', 92);
Query OK, 4 rows affected (0.05 sec)
Records: 4 Duplicates: 0 Warnings: 0
```

```
mysql> SELECT * FROM Teacher;
+-----+-----+
| TeacherID | TeacherName |
+-----+-----+
| T01       | Sonu        |
| T02       | Pranathi    |
+-----+-----+
2 rows in set (0.00 sec)

mysql> SELECT * FROM Class;
+-----+-----+-----+
| ClassID | ClassName | TeacherID |
+-----+-----+-----+
| C01     | 10-A     | T01       |
| C02     | 10-B     | T02       |
+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> SELECT * FROM Student;
+-----+-----+-----+
| StudentID | StudentName | ClassID |
+-----+-----+-----+
| S01       | Ramesh      | C01     |
| S02       | Aishu       | C01     |
| S03       | Lehan       | C02     |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT * FROM Subject;
+-----+-----+
| SubjectID | SubjectName |
+-----+-----+
| SUB01     | DBMS        |
| SUB02     | Mathematics |
| SUB03     | Python      |
+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT * FROM Enrollment;
+-----+-----+-----+
| StudentID | SubjectID | Marks |
+-----+-----+-----+
| S01       | SUB01     | 85     |
| S01       | SUB02     | 90     |
| S02       | SUB01     | 88     |
| S03       | SUB03     | 92     |
+-----+-----+-----+
4 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM Enrollment;
+-----+-----+-----+
| StudentID | SubjectID | Marks |
+-----+-----+-----+
| S01       | SUB01     | 85     |
| S01       | SUB02     | 90     |
| S02       | SUB01     | 88     |
| S03       | SUB03     | 92     |
+-----+-----+-----+
4 rows in set (0.00 sec)

mysql> SELECT s.StudentID, s.StudentName, c.ClassName, t.TeacherName, sub.SubjectName, e.Marks FROM Enrollment e JOIN Student s ON e.StudentID = s.StudentID
JOIN Class c ON s.ClassID = c.ClassID JOIN Teacher t ON c.TeacherID = t.TeacherID JOIN Subject sub ON e.SubjectID = sub.SubjectID;
+-----+-----+-----+-----+-----+-----+
| StudentID | StudentName | ClassName | TeacherName | SubjectName | Marks |
+-----+-----+-----+-----+-----+-----+
| S01       | Ramesh      | 10-A     | Sonu        | DBMS        | 85     |
| S01       | Ramesh      | 10-A     | Sonu        | Mathematics | 90     |
| S02       | Aishu       | 10-A     | Sonu        | DBMS        | 88     |
| S03       | Lehan       | 10-B     | Pranathi    | Python      | 92     |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

Given Poorly Normalized Schema

Assume the School ERP stores:

SchoolERP(StudentID, StudentName, ClassID, ClassName, TeacherID, TeacherName, SubjectID, SubjectName, Marks, ParentID, ParentName)

1. Functional Dependencies

The important functional dependencies are:

- **StudentID → StudentName, ParentID**
- **ParentID → ParentName**
- **ClassID → ClassName**
- **TeacherID → TeacherName**

- **SubjectID** → **SubjectName**
- **(StudentID, SubjectID)** → **Marks**

Assume a student can study many subjects and receive one mark for each subject.

2. Candidate Key

A student's marks are identified by the combination of:

(StudentID, SubjectID)

Therefore:

Candidate Key = (StudentID, SubjectID)

3. Identify Partial Dependencies

Since the key is (StudentID, SubjectID):

- **StudentID** → **StudentName, ParentID**
- **SubjectID** → **SubjectName**

These are **partial dependencies** because they depend on only part of the composite key.

Therefore, the original relation is **not in 2NF**.

4. Identify Transitive Dependencies

We have:

StudentID → **ParentID**

and

ParentID → **ParentName**

Therefore:

StudentID → **ParentID** → **ParentName**

This is a **transitive dependency**.

Similarly, teacher and class information should be stored independently rather than repeatedly with every student-subject record.

5. Decomposition

Student

Student(StudentID, StudentName, ParentID, ClassID)

FD:

StudentID → **StudentName, ParentID, ClassID**

Parent

Parent(ParentID, ParentName)

FD:

ParentID $\rightarrow$ ParentName

Class

Class(ClassID, ClassName, TeacherID)

FD:

ClassID $\rightarrow$ ClassName, TeacherID

Teacher

Teacher(TeacherID, TeacherName)

FD:

TeacherID $\rightarrow$ TeacherName

Subject

Subject(SubjectID, SubjectName)

FD:

SubjectID $\rightarrow$ SubjectName

StudentSubject

StudentSubject(StudentID, SubjectID, Marks)

FD:

(StudentID, SubjectID) $\rightarrow$ Marks

6. Check BCNF

A relation is in **BCNF** if every determinant is a superkey.

Student

StudentID $\rightarrow$ StudentName, ParentID, ClassID

StudentID is the key.

BCNF ✓

Parent

ParentID $\rightarrow$ ParentName

ParentID is the key.

BCNF ✓

Class

ClassID $\rightarrow$ ClassName, TeacherID

ClassID is the key.

BCNF ✓

Teacher

TeacherID $\rightarrow$ TeacherName

TeacherID is the key.

BCNF ✓

Subject

SubjectID $\rightarrow$ SubjectName

SubjectID is the key.

BCNF ✓

StudentSubject

(StudentID, SubjectID) $\rightarrow$ Marks

(StudentID, SubjectID) is the key.

BCNF ✓

7. Final BCNF Design

Student

StudentID (PK)

StudentName

ParentID (FK)

ClassID (FK)

Parent

ParentID (PK)

ParentName

Class

ClassID (PK)
ClassName
TeacherID (FK)

Teacher

TeacherID (PK)
TeacherName

Subject

SubjectID (PK)
SubjectName

StudentSubject

StudentID (PK, FK)
SubjectID (PK, FK)
Marks

8. Justification

The original School ERP relation contains **partial dependencies**, such as $\text{StudentID} \rightarrow \text{StudentName}$, when the key is $(\text{StudentID}, \text{SubjectID})$. It also contains **transitive dependencies**, such as $\text{StudentID} \rightarrow \text{ParentID} \rightarrow \text{ParentName}$. These dependencies cause **update, insertion, and deletion anomalies**.

The decomposition separates students, parents, classes, teachers, subjects, and student marks into independent relations. In every final relation, the determinant of each functional dependency is a **candidate key/superkey**. Therefore, the resulting design satisfies **BCNF**.

Final Answer

BCNF Relations:

1. Student(StudentID, StudentName, ParentID, ClassID)
2. Parent(ParentID, ParentName)
3. Class(ClassID, ClassName, TeacherID)
4. Teacher(TeacherID, TeacherName)
5. Subject(SubjectID, SubjectName)
6. StudentSubject(StudentID, SubjectID, Marks)

Result: The School ERP is fully normalized up to **BCNF**, with reduced redundancy and elimination of major insertion, deletion, and update anomalies.

10. Given a movie streaming platform schema, identify join dependencies and decompose to 5NF with lossless-join verification.

```
mysql> CREATE DATABASE MovieDB;USE MovieDB;
Query OK, 1 row affected (0.05 sec)

Database changed
mysql> CREATE TABLE Movie ( MovieID VARCHAR(10) PRIMARY KEY, MovieTitle VARCHAR(100) );
Query OK, 0 rows affected (0.08 sec)

mysql> CREATE TABLE Actor ( ActorID VARCHAR(10) PRIMARY KEY, ActorName VARCHAR(50) );
Query OK, 0 rows affected (0.08 sec)

mysql> CREATE TABLE Platform ( PlatformID VARCHAR(10) PRIMARY KEY, PlatformName VARCHAR(50) );
Query OK, 0 rows affected (0.05 sec)

mysql> CREATE TABLE Movie_Actor ( MovieID VARCHAR(10), ActorID VARCHAR(10), PRIMARY KEY (MovieID, ActorID), FOREIGN KEY (MovieID) REFERENCES Movie(MovieID)
FOREIGN KEY (ActorID) REFERENCES Actor(ActorID) );
Query OK, 0 rows affected (0.18 sec)

mysql> CREATE TABLE Movie_Platform ( MovieID VARCHAR(10), PlatformID VARCHAR(10), PRIMARY KEY (MovieID, PlatformID), FOREIGN KEY (MovieID) REFERENCES Movie
(MovieID), FOREIGN KEY (PlatformID) REFERENCES Platform(PlatformID) );
Query OK, 0 rows affected (0.10 sec)

mysql> CREATE TABLE Actor_Platform ( ActorID VARCHAR(10), PlatformID VARCHAR(10), PRIMARY KEY (ActorID, PlatformID), FOREIGN KEY (ActorID) REFERENCES Actor
(ActorID), FOREIGN KEY (PlatformID) REFERENCES Platform(PlatformID) );
Query OK, 0 rows affected (0.10 sec)

mysql> INSERT INTO Movie VALUES ('M01', 'Inception'), ('M02', 'Interstellar'), ('M03', 'Avatar');
Query OK, 3 rows affected (0.09 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Actor VALUES ('A01', 'Leonardo DiCaprio'), ('A02', 'Matthew McConaughey'), ('A03', 'Sam Worthington');
Query OK, 3 rows affected (0.05 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Platform VALUES ('P01', 'Netflix'), ('P02', 'Amazon Prime'), ('P03', 'Disney+');
Query OK, 3 rows affected (0.05 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Movie_Actor VALUES ('M01', 'A01'), ('M02', 'A02'), ('M03', 'A03');
Query OK, 3 rows affected (0.05 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Movie_Platform VALUES ('M01', 'P01'), ('M01', 'P02'), ('M02', 'P01'), ('M03', 'P03');
Query OK, 4 rows affected (0.05 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Actor_Platform VALUES
```

```
Records: 4 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Actor_Platform VALUES
-> ('A01', 'P01'),
-> ('A01', 'P02'),
-> ('A02', 'P01'),
-> ('A03', 'P03');
Query OK, 4 rows affected (0.08 sec)
Records: 4 Duplicates: 0 Warnings: 0
```

```
mysql> SELECT * FROM Movie;
+-----+-----+
| MovieID | MovieTitle |
+-----+-----+
| M01     | Inception  |
| M02     | Interstellar |
| M03     | Avatar     |
+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT * FROM Actor;
+-----+-----+
| ActorID | ActorName |
+-----+-----+
| A01     | Leonardo DiCaprio |
| A02     | Matthew McConaughey |
| A03     | Sam Worthington |
+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT * FROM Platform;
+-----+-----+
| PlatformID | PlatformName |
+-----+-----+
| P01        | Netflix      |
| P02        | Amazon Prime |
| P03        | Disney+     |
+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT * FROM Movie_Actor;
+-----+-----+
| MovieID | ActorID |
+-----+-----+
| M01     | A01     |
| M02     | A02     |
| M03     | A03     |
+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT * FROM Movie_Platform;
+-----+-----+
| MovieID | PlatformID |
+-----+-----+
| M01     | P01        |
| M02     | P01        |
+-----+-----+
2 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM Actor_Platform;
+-----+-----+
| ActorID | PlatformID |
+-----+-----+
| A01     | P01        |
| A02     | P01        |
| A03     | P03        |
+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT m.MovieID, m.MovieTitle, a.ActorID, a.ActorName, p.PlatformID, p.PlatformName FROM Movie m JOIN Movie_Actor ma ON m.MovieID = ma.MovieID JOIN
Actor a ON ma.ActorID = a.ActorID JOIN Movie_Platform mp ON m.MovieID = mp.MovieID JOIN Platform p ON mp.PlatformID = p.PlatformID JOIN Actor_Platform ap ON
a.ActorID = ap.ActorID AND p.PlatformID = ap.PlatformID;
+-----+-----+-----+-----+-----+-----+
| MovieID | MovieTitle | ActorID | ActorName | PlatformID | PlatformName |
+-----+-----+-----+-----+-----+-----+
| M01     | Inception  | A01     | Leonardo DiCaprio | P01        | Netflix      |
| M01     | Inception  | A01     | Leonardo DiCaprio | P02        | Amazon Prime |
| M02     | Interstellar | A02     | Matthew McConaughey | P01        | Netflix      |
| M03     | Avatar     | A03     | Sam Worthington | P03        | Disney+     |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

Given Relation

Assume the movie streaming platform has the relation:

Streaming(MovieID, ActorID, PlatformID)

Where:

- MovieID identifies a movie.
- ActorID identifies an actor.
- PlatformID identifies a streaming platform.

A movie can have multiple actors and can be available on multiple platforms.

1. Identify the Join Dependency

Suppose the relationship between **Movie**, **Actor**, and **Platform** can be represented independently.

The relation has the join dependency:

(MovieID, ActorID), (MovieID, PlatformID), (ActorID, PlatformID)

This means the complete Streaming relation can be reconstructed by joining the following three projections:

- Movie–Actor
- Movie–Platform
- Actor–Platform

Formally:

Streaming = $\pi(\text{MovieID}, \text{ActorID})(\text{Streaming}) \bowtie \pi(\text{MovieID}, \text{PlatformID})(\text{Streaming}) \bowtie \pi(\text{ActorID}, \text{PlatformID})(\text{Streaming})$

2. Why 5NF is Required

A relation is in **5NF (Project-Join Normal Form)** when every non-trivial join dependency is implied by the candidate keys.

If the three-way relationship contains independent relationships, storing all three attributes together can create unnecessary redundancy.

For example, suppose:

- Movie M1 has Actor A1.
- Movie M1 is available on Platform P1.
- Actor A1 works with Platform P1.

Storing these relationships independently can produce unnecessary combinations in the original relation.

Therefore, we decompose the relation.

3. 5NF Decomposition

Movie–Actor

MovieActor(MovieID, ActorID)

This stores which actors are associated with which movies.

Movie–Platform

MoviePlatform(MovieID, PlatformID)

This stores which movies are available on which platforms.

Actor–Platform

ActorPlatform(ActorID, PlatformID)

This stores the actor-platform relationship.

4. Final 5NF Schema

MovieActor

MovieID

ActorID

MoviePlatform

MovieID

PlatformID

ActorPlatform

ActorID

PlatformID

Each relation contains only a binary relationship and therefore eliminates the non-trivial three-way join dependency.

5. Lossless-Join Verification

To verify lossless decomposition, join the three decomposed relations:

MovieActor ⋈ MoviePlatform ⋈ ActorPlatform

Formally:

Streaming = MovieActor ⋈ MoviePlatform ⋈ ActorPlatform

If this join produces exactly the original Streaming relation, the decomposition is **lossless**.

Example

Suppose:

MovieActor

M1 A1

M1 A2

MoviePlatform

M1 P1

M1 P2

ActorPlatform

A1 P1

A2 P2

The three relations preserve the relationships without requiring redundant storage of the complete three-way combination.

6. Lossless-Join Condition

The decomposition is considered lossless when:

$$\pi(\text{MovieID}, \text{ActorID})(R) \bowtie \pi(\text{MovieID}, \text{PlatformID})(R) \bowtie \pi(\text{ActorID}, \text{PlatformID})(R) = R$$

Therefore, no information is lost when the decomposed relations are joined.

7. Final Result

Original Relation

Streaming(MovieID, ActorID, PlatformID)

Join Dependency

(MovieID, ActorID), (MovieID, PlatformID), (ActorID, PlatformID)

5NF Decomposition

1. MovieActor(MovieID, ActorID)
2. MoviePlatform(MovieID, PlatformID)
3. ActorPlatform(ActorID, PlatformID)

Conclusion

The original movie streaming relation contains a **non-trivial join dependency** involving Movie, Actor, and Platform. Decomposing it into the three binary relations removes the join dependency from the individual relations. The original relation can be reconstructed through their natural join, so the decomposition is **lossless** and satisfies the requirements of 5NF (**Project-Join Normal Form**).